# Record - Week 05

Topic: Full Stack Integration (MERN)

Name: _____ Roll No: _____ Date: _____

## Aim

To integrate a React frontend with an Express and MongoDB backend into a working full stack application.

## Software Requirements

Node.js, MongoDB, React (Vite), VS Code, Chrome DevTools

## Procedure

1. Set up server/ with Express, Mongoose and CORS.
2. Create the Todo model and CRUD routes.
3. Set up client/ with Vite and React.
4. Call the API from App.jsx using fetch.
5. Run both servers and verify the flow in the Network tab.

# Program

### server/server.js

```
require("dotenv").config();
const express = require("express");
const cors = require("cors");
const mongoose = require("mongoose");

const app = express();
app.use(cors());
app.use(express.json());

const Todo = mongoose.model(
  "Todo",
  new mongoose.Schema({
    title: { type: String, required: true },
    done: { type: Boolean, default: false },
  })
);

app.get("/api/todos", async (req, res) => res.json(await Todo.find()));

app.post("/api/todos", async (req, res) => {
  const todo = await Todo.create({ title: req.body.title });
  res.status(201).json(todo);
});

app.put("/api/todos/:id", async (req, res) => {
  const todo = await Todo.findByIdAndUpdate(req.params.id, req.body, { new: true });
  res.json(todo);
});

app.delete("/api/todos/:id", async (req, res) => {
  await Todo.findByIdAndDelete(req.params.id);
  res.json({ message: "deleted" });
});

mongoose
  .connect(process.env.MONGO_URI)
  .then(() => app.listen(process.env.PORT || 5000, () => console.log("API running")))
  .catch((err) => console.error(err));
```

### client/src/App.jsx

```
import { useEffect, useState } from "react";

const API = "http://localhost:5000/api/todos";

export default function App() {
  const [todos, setTodos] = useState([]);
  const [title, setTitle] = useState("");

  const load = async () => setTodos(await (await fetch(API)).json());

  useEffect(() => { load(); }, []);

  const add = async () => {
    if (!title.trim()) return;
    await fetch(API, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ title }),
    });
    setTitle("");
    load();
  };

  const remove = async (id) => {
    await fetch(`${API}/${id}`, { method: "DELETE" });
    load();
  };

  return (
    <div style={{ maxWidth: 420, margin: "2rem auto", fontFamily: "sans-serif" }}>
      <h1>MERN Todo</h1>
```

```
      <input value={title} onChange={(e) => setTitle(e.target.value)} placeholder="New todo" />
      <button onClick={add}>Add</button>
      <ul>
        {todos.map((t) => (
          <li key={t._id}>
            {t.title} <button onClick={() => remove(t._id)}>x</button>
          </li>
        ))}
      </ul>
    </div>
  );
}
```

## Output

Fig: frontend ui output

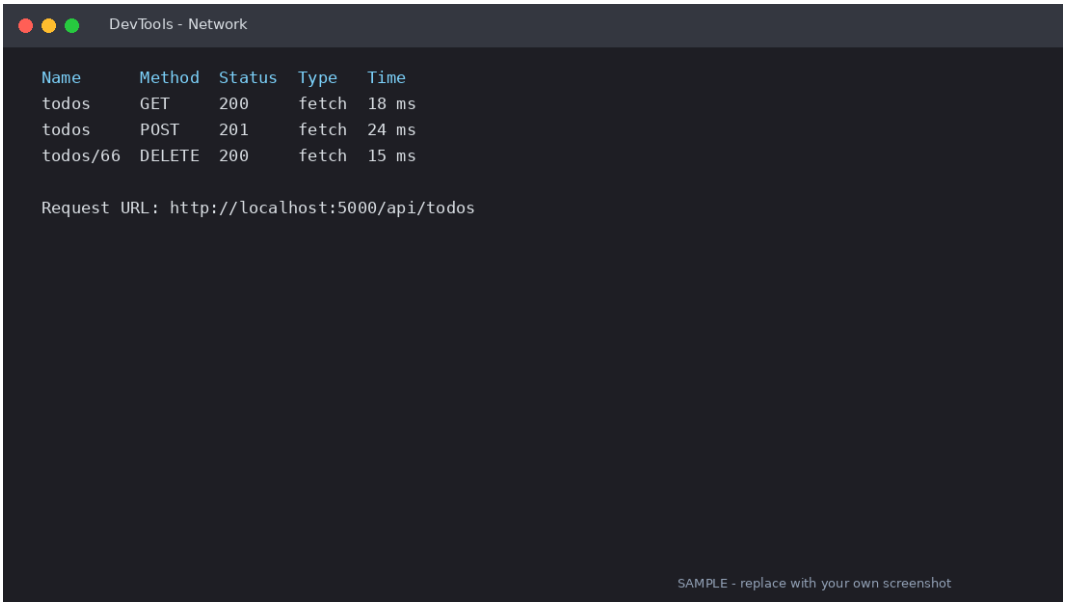


Fig: network tab api call

## Result

The program was executed successfully and the expected output was obtained.